Семинар 2. Часы и время в распределенных системах

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Материалы семинаров и домашек появились на GitHub
- Есть вероятность, что первая домашка будет посвящена часам! (о них – сегодня)

Содержание

1. Время в распределенных системах
2. Часы реального времени и алгоритмы синхронизации
3. Монотонные часы
4. Частично упорядоченное время и логические часы

Время как проблема

- В распределённой системе нет глобальных часов: разные узлы, разные ядра процессора, разный «пульс времени»
- Время передачи сообщений в общем случае неизвестно и может быть большим (пример с ping)
- Еще хуже – физические часы дрейфуют ($\pm\epsilon$)

Вопрос: как определить, какое событие произошло раньше? В идеальном случае хочется:

- Монотонность – «время течет всегда вперед»
- Согласованность – если a произошло до b, то $t(a) < t(b)$
- Уникальность – разные события имеют различимые метки времени

Два типа времени

Тотально упорядоченно (total order, linear, wall-clock, linerializble)

Любые два события сравнимы $<$: до или после (иррефлексивное, транзитивное, антисимметричное).

Частично упорядоченное (partial order, causal, logical, concurrent)

Некоторые события несравнимы $<$: до, после или конкурентно (иррефлексивное, транзитивное, антисимметричное).

Физическое время $\Rightarrow$ тотальный порядок
Причинность $\Rightarrow$ частичный порядок

Содержание

1. Время в распределенных системах
2. Часы реального времени и алгоритмы синхронизации
3. Монотонные часы
4. Частично упорядоченное время и логические часы

Real-Time Clock

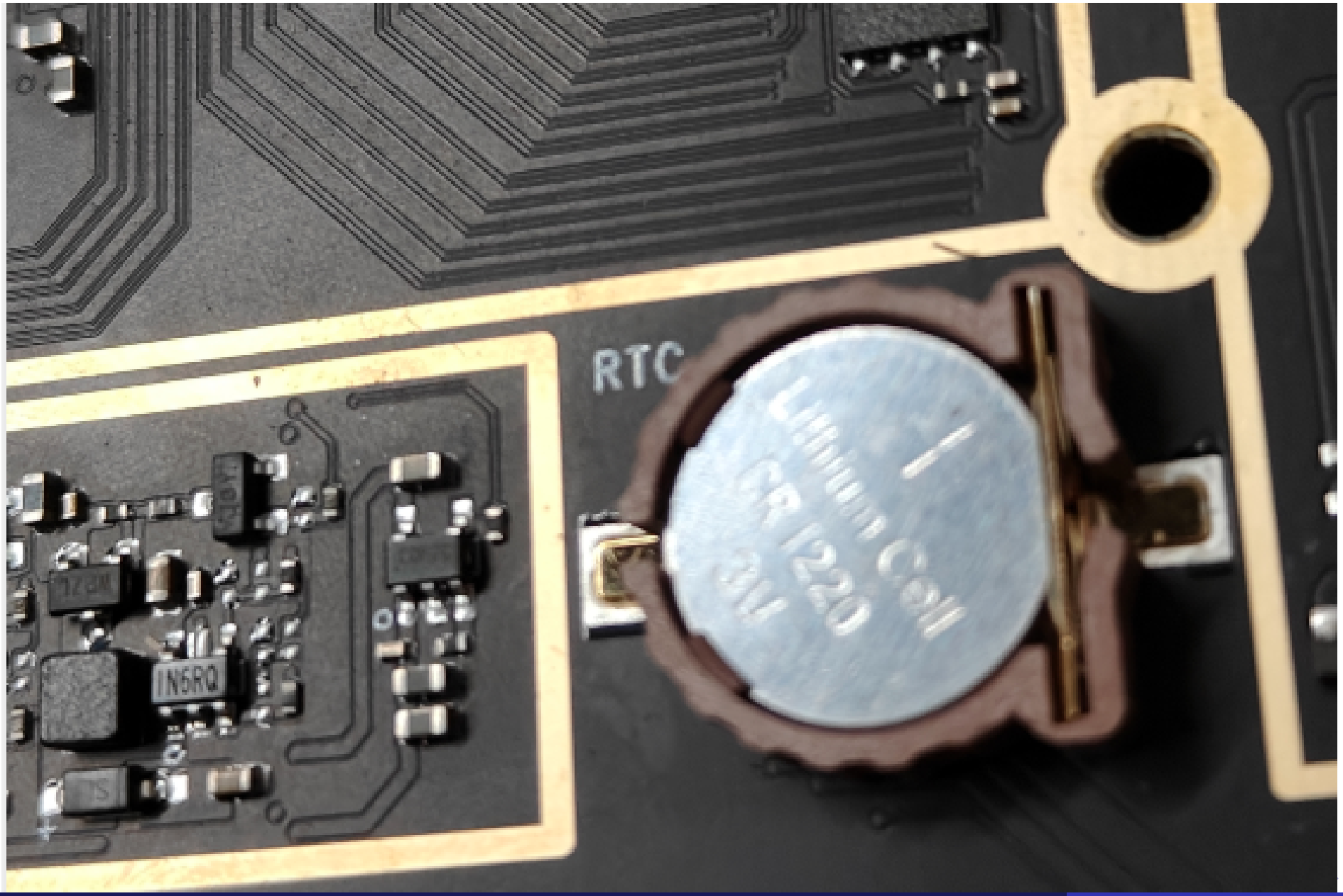
Модель реального времени (физические часы) – это функция $C(t)$, которая отображает реальное время t в метку времени C.

$$C(t) = t + \epsilon(t)$$

где $\epsilon(t)$ — ошибка.
Два параметра:

- **Drift** — отклонение скорости (разность тактовой частоты от эталона)
- **Skew** — разница между часами (относительный сдвиг)

Как дела обстояли на материнской плате?



Синхронизация часов

Cristian's algorithm (1989)

- Осуществляем Round-Trip с запросом времени
- Вычисляем сдвиг как временной сдвиг между клиентом и сервером: $RTT/2$
- В чем недостаток алгоритма?

Berkeley algorithm (1989)

- Делегируем быть «часовщиком» одному узлу системы (через leader election)
- Как в прошлом алгоритме, собираем и оцениваем время на узлах (включая себя) через Round-Trip
- Усредняем и сообщаем всем узлам, какой offset им надо к себе прибавить

Алгоритм Кристиана

Алгоритм Кристиана – алгоритм синхронизации времени между клиентом и временным сервером.
Идея: клиент отправляет запрос серверу времени и корректирует своё локальное время на основе полученного ответа.

Процесс:

1. Клиент отправляет запрос серверу времени в момент T_0 (по его часам)
2. Сервер отвечает со своим временем T_s
3. Клиент получает ответ в момент T_1 (по его часам)

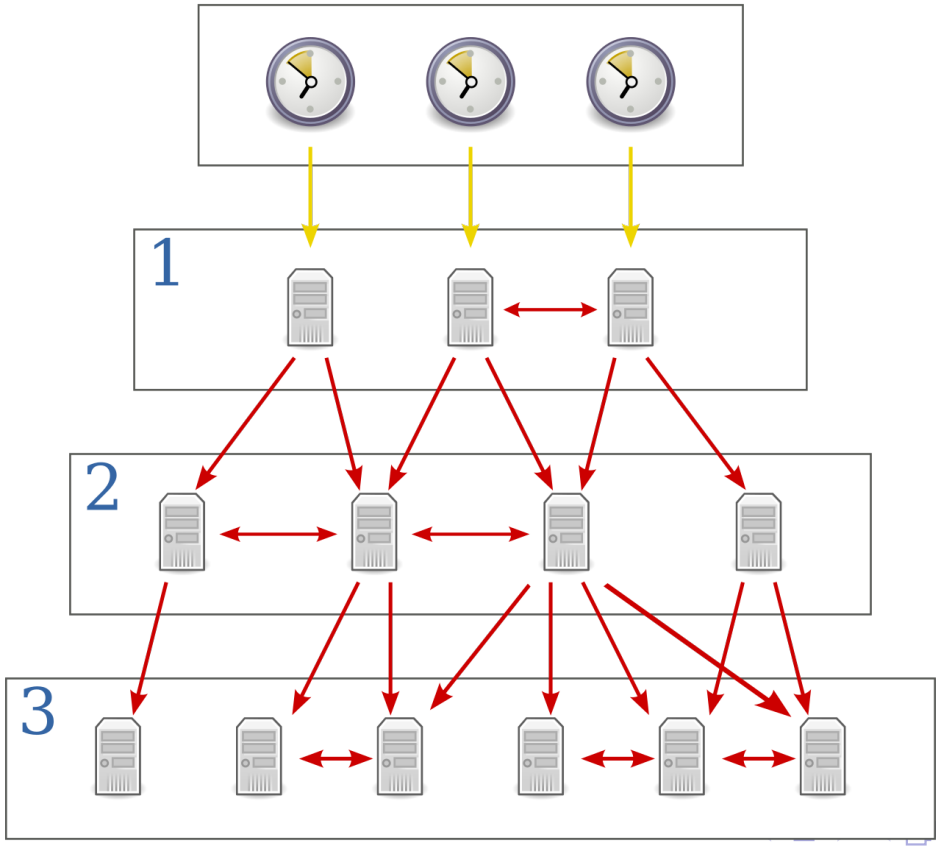
Оценка задержки и коррекция:
Время в пути туда и обратно: $RTT = T_1 - T_0$
Предполагаемое время в пути в одну сторону: $\delta = \frac{RTT}{2}$
Скорректированное время сервера: $T_s + \delta$

Оценка ошибки: Максимальная ошибка синхронизации: $\pm\delta = \pm\frac{RTT}{2}$

NTP

Network Time Protocol – семейство алгоритмов с поддержкой масштабируемости:

- Round-Trip с оценкой корректировки (алгоритм Marzullo)
- Иерархия страт (stratum 0 – эталонные часы)
- Точность – миллисекунды (Интернет), микросекунды (LAN)



TrueTime (Google Spanner)

Идея:

$$TT.now() = [earliest, latest]$$

Время задаётся интервалом неопределённости.
Позволяет:

- обеспечить внешнюю консистентность
- реализовать commit-wait

Содержание

1. Время в распределенных системах
2. Часы реального времени и алгоритмы синхронизации
3. Монотонные часы
4. Частично упорядоченное время и логические часы

Monotonic Clock

Требование:

$$t_2 > t_1 \Rightarrow C(t_2) \geq C(t_1)$$

Используется для:

- измерения интервалов
- таймаутов
- backoff

Модель времени в C++

`std::chrono::system_clock`

- отражает wall-clock
- может изменяться назад

`std::chrono::steady_clock`

- монотонный
- не связан с календарным временем
- не гарантирована синхронизация между машинами

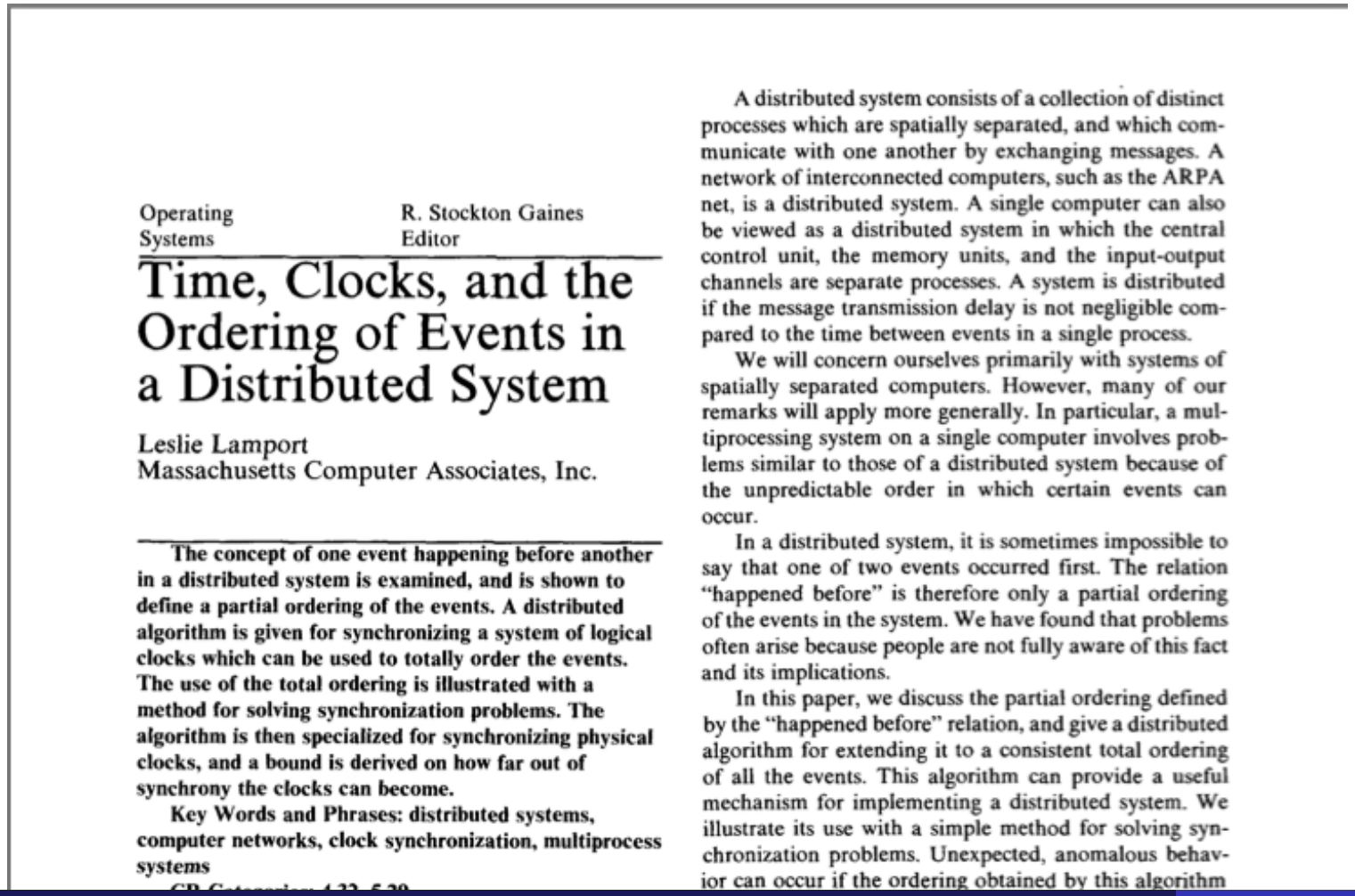
Содержание

1. Время в распределенных системах
2. Часы реального времени и алгоритмы синхронизации
3. Монотонные часы
4. Частично упорядоченное время и логические часы

Lamport (1978)

Статья: *Time, Clocks, and the Ordering of Events in a Distributed System*
Главная идея:

Время = причинность



Operating Systems — R. Stockton Gaines, Editor

Time, Clocks, and the Ordering of Events in a Distributed System

Leslie Lamport
Massachusetts Computer Associates, Inc.

The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events. A distributed algorithm is given for synchronizing a system of logical clocks which can be used to totally order the events. The use of the total ordering is illustrated with a method for solving synchronization problems. The algorithm is then specialized for synchronizing physical clocks, and a bound is derived on how far out of synchrony the clocks can become.

Key Words and Phrases: distributed systems, computer networks, clock synchronization, multiprocess systems

A distributed system consists of a collection of distinct processes which are spatially separated, and which communicate with one another by exchanging messages. A network of interconnected computers, such as the ARPA net, is a distributed system. A single computer can also be viewed as a distributed system in which the central control unit, the memory units, and the input-output channels are separate processes. A system is distributed if the message transmission delay is not negligible compared to the time between events in a single process.

We will concern ourselves primarily with systems of spatially separated computers. However, many of our remarks will apply more generally. In particular, a multiprocessing system on a single computer involves problems similar to those of a distributed system because of the unpredictable order in which certain events can occur.

In a distributed system, it is sometimes impossible to say that one of two events occurred first. The relation "happened before" is therefore only a partial ordering of the events in the system. We have found that problems often arise because people are not fully aware of this fact and its implications.

In this paper, we discuss the partial ordering defined by the "happened before" relation, and give a distributed algorithm for extending it to a consistent total ordering of all the events. This algorithm can provide a useful mechanism for implementing a distributed system. We illustrate its use with a simple method for solving synchronization problems. Unexpected, anomalous behavior can occur if the ordering obtained by this algorithm

Отношение happens-before

Определение:
$a \to b$, если:

1. События в одном процессе и a раньше b
2. a — отправка, b — получение сообщения
3. Транзитивность

Тотально упорядоченное время

$$\forall a, b : a \to b \Leftrightarrow t(a) < t(b)$$

Частично упорядоченное время

$$\forall a, b : a \to b \Rightarrow t(a) < t(b)$$

Часы Лампорта

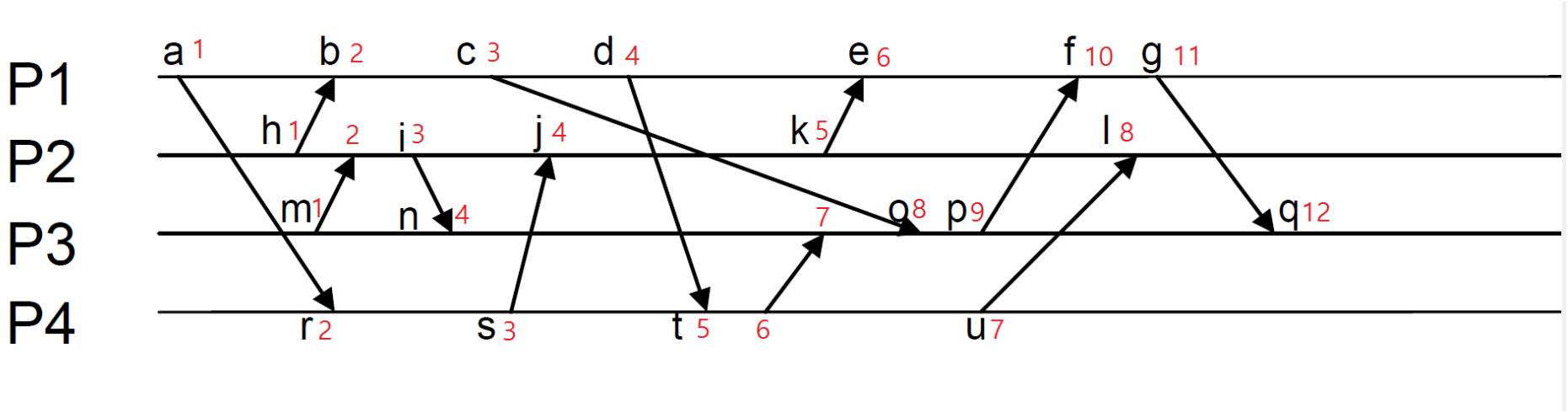
Интуиция:

- "ничто на Земле не проходит бесследно"
- если произошло B, и A happens-before B, то это должно привести к «утечке» времени, т.е. $L(A) < L(B)$

Алгоритм для каждого процесса P с локальной переменной C:

1. Когда хочу создать событие:
   - Добавлю в событие метку, соответствующую моим текущим часам: $L := C$
   - Атомарно инкрементирую свои часы на единицу: $C := C + 1$
2. Когда получаю чужое событие:
   - Обновляю свои часы: $C := \max(C, C_m) + 1$

Иллюстрация часов Лампорта



(!) Часы Лампорта не позволяет причинную связь между событиями! Они лишь позволяют понять следующее:

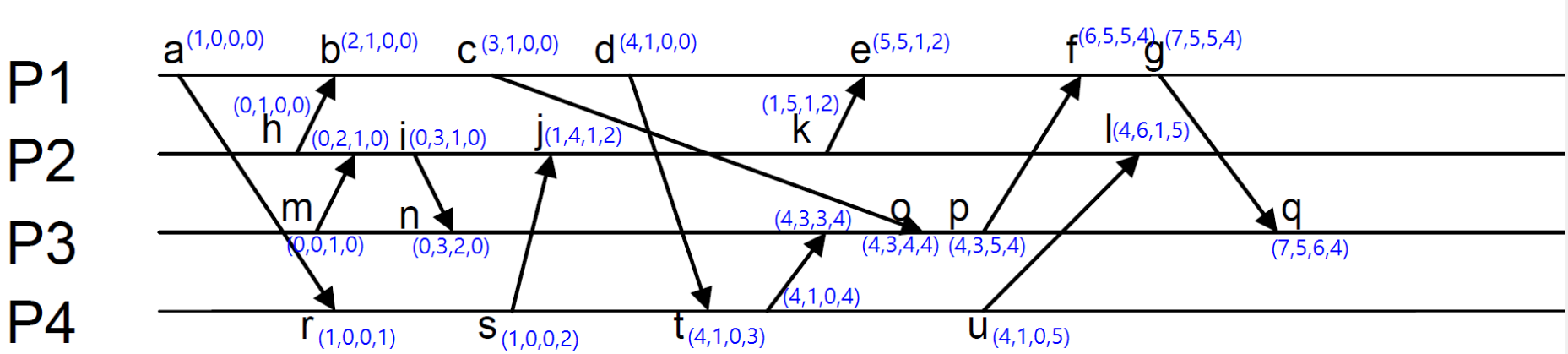
$$L(A) \geq L(B) \Rightarrow A \not\rightarrow B$$

Векторные часы

Каждый процесс хранит вектор размера N с текущей известной меткой времени, происходящего на каждом из процессов:

1. Когда хочу создать событие:
   - Добавлю в событие метку, соответствующую моим текущим часам: $L := C[i]$
   - Атомарно инкрементирую свои часы на единицу: $C[i] := C[i] + 1$
2. Когда получаю чужое событие:
   - Обновляю свои часы: $\forall j : C[j] := \max(C[j], C_m[j])$
   - Атомарно инкрементирую свои часы на единицу: $C[i] := C[i] + 1$

Иллюстрация векторных часов



a b c

a c b

c a b

1 (1,0,0), (1,1,1), (0,1,0)

Как определяется отношение happens-before между векторными часами A и B?

$$A \to B \Leftrightarrow \forall i : A[i] \leq B[i] \text{ и } \exists i : A[i] < B[i]$$

$$A \parallel B \Leftrightarrow \exists i : A[i] > B[i] \text{ и } \exists i : A[i] < B[i]$$

Минимальность

Векторные часы — минимальное точное представление частичного порядка в общем случае (Charron-Bost, "Concerning the size of clocks", 1990)

- Causal consistency: доставка сообщений в порядке причинности
- CRDT: операции, которые могут быть применены в любом порядке, но результат будет одинаковым
- Отслеживание конфликтов: два параллельных события могут привести к конфликту, который нужно разрешить
- Distributed debugging/tracing/thread sanitizer

Итоги

- Время – то, что позволяет нам рассуждать о событиях внутри распределенной системы
- В зависимости от порядка событий мы можем говорить о разных состояниях системы
- Физическое время – привычно нам, но оно не обладает монотонностью
- Логическое время – относительная конструкция, но с ней можно рассуждать о причинности

Блогопосты, чтобы разобраться лучше

1. Kevin Sookocheff. Lamport Clocks
2. Martin Fowler. Lamport Clocks
3. GeeksforGeeks. Lamport's Logical Clock
4. Michael Whittaker. Lamport's Logical Clocks
5. Kevin Sookocheff. Vector Clocks
6. Balraja Subbiah. Lamport Clocks and Vector Clocks